

Занятие 6

Загрузка данных в хранилище (Data Warehouse)

6.1. Цели и задачи работы

Цель работы: реализовать загрузку очищенных данных в спроектированное хранилище, выполнить миграцию данных из плоских файлов в реляционную базу данных.

Задачи работы:

1. Установить и настроить СУБД PostgreSQL.
2. Создать базу данных и таблицы по спроектированной схеме.
3. Загрузить данные из CSV-файлов в таблицы.
4. Проверить целостность и полноту загруженных данных.
5. Создать индексы для оптимизации запросов.

6.2. Теоретические сведения

6.2.1. Выбор СУБД для хранилища

Для учебных проектов рекомендуется использовать **PostgreSQL** — свободную объектно-реляционную СУБД с широкими возможностями.

Преимущества PostgreSQL:

- Полностью бесплатная (лицензия PostgreSQL)
- Поддержка сложных запросов и оконных функций
- Хорошая производительность на данных до сотен ГБ
- Поддержка JSON для гибких схем
- Большое сообщество и документация

Альтернативы:

- **SQLite** — для очень маленьких проектов (до 1 млн строк)
- **ClickHouse** — для больших объёмов и аналитики (колоночная СУБД)
- **MySQL/MariaDB** — тоже возможны, но менее функциональны для аналитики

6.2.2. Способы загрузки данных в PostgreSQL

Способ	Команда/Инструмент	Скорость	Когда использовать
INSERT	INSERT INTO ... VALUES	Низкая	Для вставки одной записи
INSERT множественный	INSERT INTO ... VALUES (...), (...)	Средняя	Для десятков записей
COPY	COPY table FROM 'file.csv' CSV HEADER	Высокая	Для загрузки больших файлов

pgAdmin	GUI-интерфейс импорта	Средняя	Для разовой загрузки
pandas + SQLAlchemy	df.to_sql()	Средняя	Из Python-скриптов

6.2.3. Создание индексов

Индексы ускоряют выполнение запросов, но замедляют вставку данных и занимают место.

```
-- Простой индекс на поле
CREATE INDEX idx_movie_year ON dim_movie(release_year);

-- Составной индекс
CREATE INDEX idx_rating_movie_user ON fact_rating(movie_id, user_id);

-- Уникальный индекс
CREATE UNIQUE INDEX idx_movie_title ON dim_movie(title);
```

6.3. Порядок выполнения работы

Шаг 1. Установка и настройка PostgreSQL

Вариант А: Локальная установка

6. Скачайте PostgreSQL с [postgresql.org/download](https://www.postgresql.org/download)
7. Установите, запомните пароль для суперпользователя postgres
8. Установите pgAdmin (обычно идёт в комплекте)

Вариант Б: Docker (рекомендуется)

```
docker run --name postgres-ai -e POSTGRES_PASSWORD=mysecretpassword -p 5432:5432 -d postgres:14
```

Проверка установки:

```
psql -U postgres -h localhost
# или через pgAdmin
```

Шаг 2. Создание базы данных и таблиц

2.1. Создание базы данных

```
CREATE DATABASE ai_project;
\c ai_project;
```

2.2. Создание таблиц — используйте SQL-скрипты с прошлого занятия. Выполните их в pgAdmin или через psql.

2.3. Проверка создания таблиц

```
\d
SELECT count(*) FROM information_schema.tables WHERE table_schema = 'public';
```

Шаг 3. Загрузка данных через Python

Создайте Python-скрипт для загрузки данных:

```

import pandas as pd
from sqlalchemy import create_engine
import psycopg2

# Параметры подключения
DB_NAME = 'ai_project'
DB_USER = 'postgres'
DB_PASSWORD = 'mysecretpassword'
DB_HOST = 'localhost'
DB_PORT = '5432'

# Создание engine для SQLAlchemy
engine = create_engine(f'postgresql://{DB_USER}:{DB_PASSWORD}@{DB_HOST}:{DB_PORT}/{DB_NAME}')

# Загрузка очищенных данных
df = pd.read_csv('data_cleaned.csv')

# Загрузка таблицы измерений фильмов
dim_movie = df[['movie_id', 'title', 'release_year', 'genre', 'director']].drop_duplicates()
dim_movie.to_sql('dim_movie', engine, if_exists='append', index=False)
print(f"Загружено фильмов: {len(dim_movie)}")

# Загрузка таблицы измерений пользователей (если есть)
if 'user_id' in df.columns:
    dim_user = df[['user_id', 'user_name', 'age', 'country']].drop_duplicates()
    dim_user.to_sql('dim_user', engine, if_exists='append', index=False)
    print(f"Загружено пользователей: {len(dim_user)}")

# Загрузка таблицы фактов
fact_rating = df[['movie_id', 'user_id', 'rating', 'votes_count']]
fact_rating.to_sql('fact_rating', engine, if_exists='append', index=False)
print(f"Загружено оценок: {len(fact_rating)}")

```

Альтернативный способ: прямой COPY через psql

```

import psycopg2

conn = psycopg2.connect(dbname=DB_NAME, user=DB_USER, password=DB_PASSWORD,
                        host=DB_HOST)
cur = conn.cursor()
with open('dim_movie.csv', 'r') as f:
    cur.copy_expert("COPY dim_movie FROM STDIN WITH CSV HEADER", f)
conn.commit()
cur.close()
conn.close()

```

Шаг 4. Проверка целостности данных

Выполните проверочные запросы:

```

-- Проверка количества записей
SELECT 'dim_movie' AS table_name, count(*) AS count FROM dim_movie
UNION ALL
SELECT 'dim_user', count(*) FROM dim_user
UNION ALL
SELECT 'fact_rating', count(*) FROM fact_rating;

```

```

-- Проверка наличия внешних ключей (orphaned rows)
SELECT COUNT(*) FROM fact_rating f
LEFT JOIN dim_movie m ON f.movie_id = m.movie_id
WHERE m.movie_id IS NULL;

-- Проверка на NULL в ключевых полях
SELECT COUNT(*) FROM fact_rating WHERE movie_id IS NULL;

-- Базовая статистика
SELECT
    MIN(rating)      AS min_rating,
    MAX(rating)      AS max_rating,
    AVG(rating)      AS avg_rating,
    STDDEV(rating)   AS stddev_rating
FROM fact_rating;

```

Шаг 5. Создание индексов

```

-- Индексы для ускорения запросов
CREATE INDEX idx_fact_movie ON fact_rating(movie_id);
CREATE INDEX idx_fact_user ON fact_rating(user_id);
CREATE INDEX idx_fact_rating ON fact_rating(rating);

-- Составной индекс для частых фильтров
CREATE INDEX idx_fact_movie_rating ON fact_rating(movie_id, rating);

-- Анализ использования индексов
EXPLAIN ANALYZE SELECT * FROM fact_rating WHERE movie_id = 123;

```

6.4. Структура отчёта

Раздел 1. Цель работы

Раздел 2. Настройка инфраструктуры

- Параметры подключения к БД
- Версии ПО

Раздел 3. Процесс загрузки данных

- Скрипт загрузки с комментариями
- Количество загруженных записей по таблицам

Раздел 4. Проверка целостности

- Результаты проверочных запросов
- Скриншоты результатов

Раздел 5. Оптимизация

- Созданные индексы
- Результаты EXPLAIN ANALYZE

Раздел 6. Выводы

6.5. Контрольные вопросы

1. Какие способы загрузки данных в PostgreSQL существуют и какой выбрали вы?
2. Как обеспечить согласованность данных при загрузке?
3. Что такое транзакция и как её использовать при загрузке?
4. Какие индексы вы создали и почему?
5. Как проверить, что все внешние ключи ссылаются на существующие записи?

6.6. Литература

1. Кимбалл, Р. Инструментарий хранения и анализа данных (см. полное описание в занятии 5) — раздел о медленно меняющихся измерениях (SCD) и стратегиях загрузки.
2. Медленно меняющиеся измерения (SCDs) в эпоху облачного хранилища данных // Хабр. URL: <https://habr.com/ru/companies/slurm/articles/743962/> (дата обращения: 10.07.2026). Реализация SCD в облачных DWH.
3. Как медленно меняющиеся измерения помогают сохранить контекст изменений в данных // Хабр. URL: <https://habr.com/ru/articles/886918/> (дата обращения: 10.07.2026). Объяснение типов SCD.
4. Что такое DWH (КХД) и как работает корпоративное хранилище данных // Хабр. URL: <https://habr.com/ru/articles/1037496/> (дата обращения: 10.07.2026). Загрузка «снизу вверх» через витрины данных.
5. Технология проектирования хранилищ данных Data Vault 2.0 // Хабр. URL: <https://habr.com/ru/articles/850280/> (дата обращения: 10.07.2026). Альтернативная методология загрузки/моделирования.
6. Секреты построения Data Vault 2.0: эффективное хранение данных // Хабр. URL: <https://habr.com/ru/companies/cinimex/articles/857084/> (дата обращения: 10.07.2026). Практический разбор загрузки по Data Vault.